



Universidade do Minho

Escola de Engenharia

Departamento de Informática

APRENDIZAGEM AUTOMÁTICA

Machine Learning

Clustering

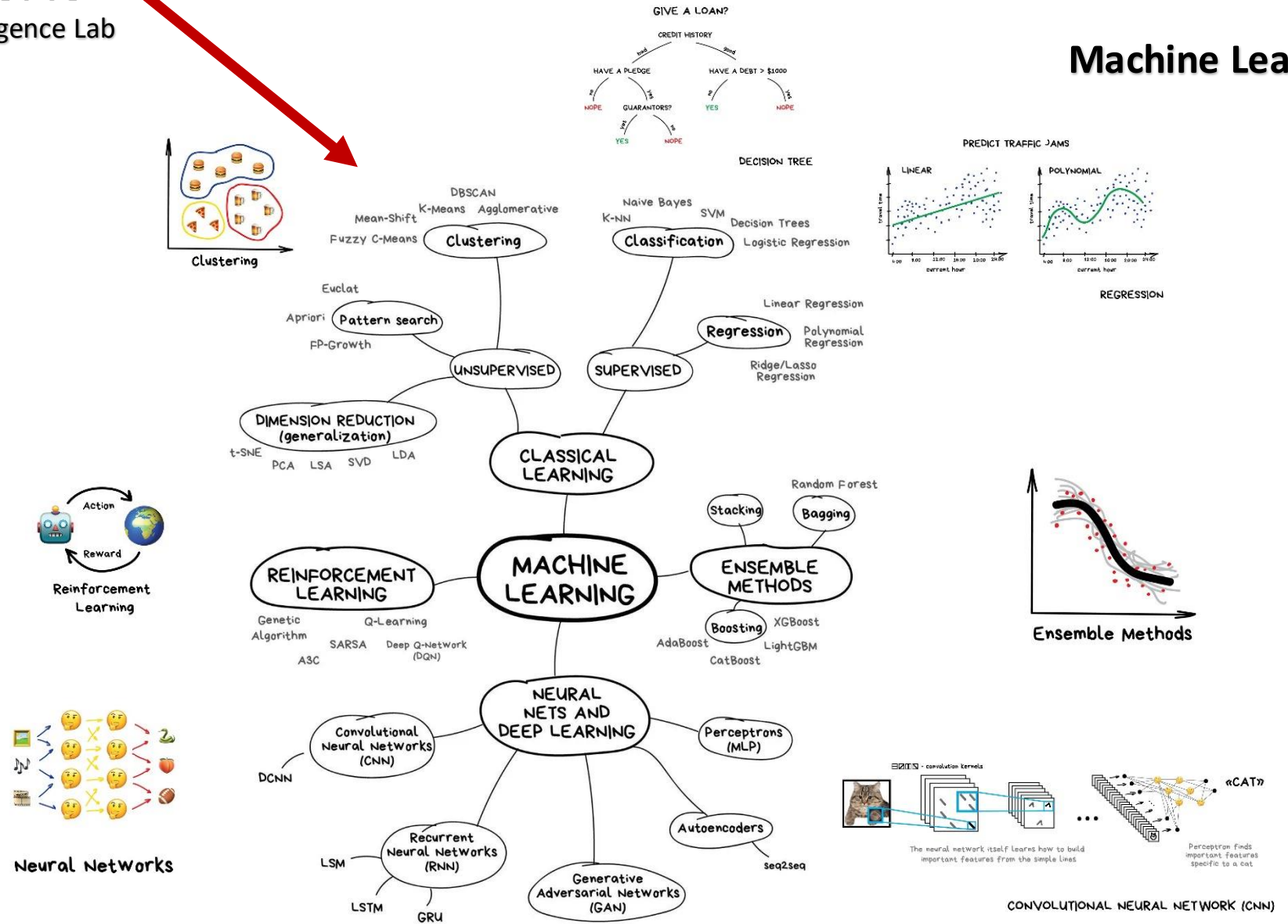
MESTRADO EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA

MESTRADO integrado EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA

Inteligência Artificial para as Telecomunicações

2025/2026

Machine Learning



- **Unsupervised Learning:**
 - **Segmentation/Clustering:**
 - Applications;
 - Main segmentation methods;
 - Partitioning algorithms;
 - Hierarchisation algorithms;
 - Other algorithms;
 - Examples.

What is Segmentation?



(gathering opinions in class)

What is Segmentation/Clustering?

- Data segmentation/clustering is the process of partitioning a set of data into smaller segments/clusters that group together similar sets of data.



What is Segmentation/Clustering?

- A Segment/Cluster is a collection of values/objects that:
 - are similar to each other within the same segment;
 - are different from the values/objects in other segments:



What is Segmentation/Clustering?

- A Segment/Cluster is a collection of values/objects that:
 - are similar to each other within the same segment;
 - are different from the values/objects in other segments:



- Measures of similarity:
 - Euclidean or Manhattan distance, for continuous attributes;
 - Jacqard coefficient, for discrete/binary attributes;
 - etc.

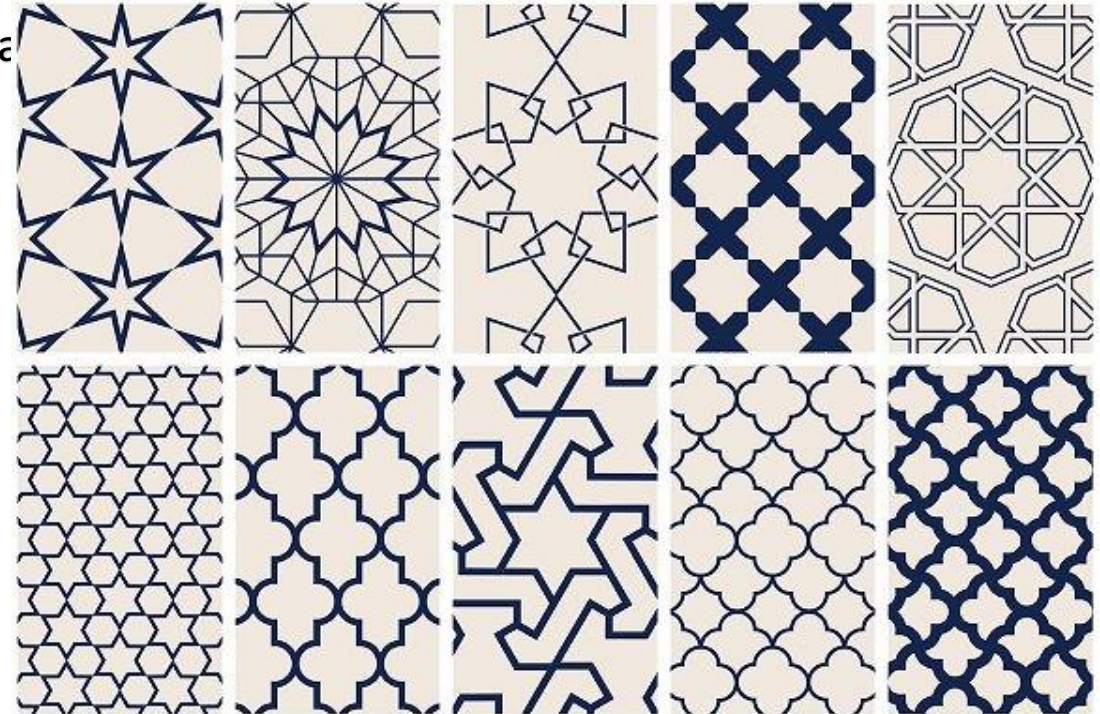
Segmentation Applications

- As a tool in itself, to search "inside" the data, about the distribution of its values;
 - As one of the pre-processing phases, in order to organise the data to be submitted to other algorithms;
 - In pattern matching problems;
 - In image processing;
 - In economic market research;
 - among others.
- 



Using Segmentation

- Segment detection is useful
- when we suspect the **existence of "natural" groupings**, which may represent groups of customers, products or goods that share (a lot of) information;
- when there are **many different patterns** in the data, making it difficult to identify a particular pattern;
 - creating similar segments reduces the complexity of the problem.

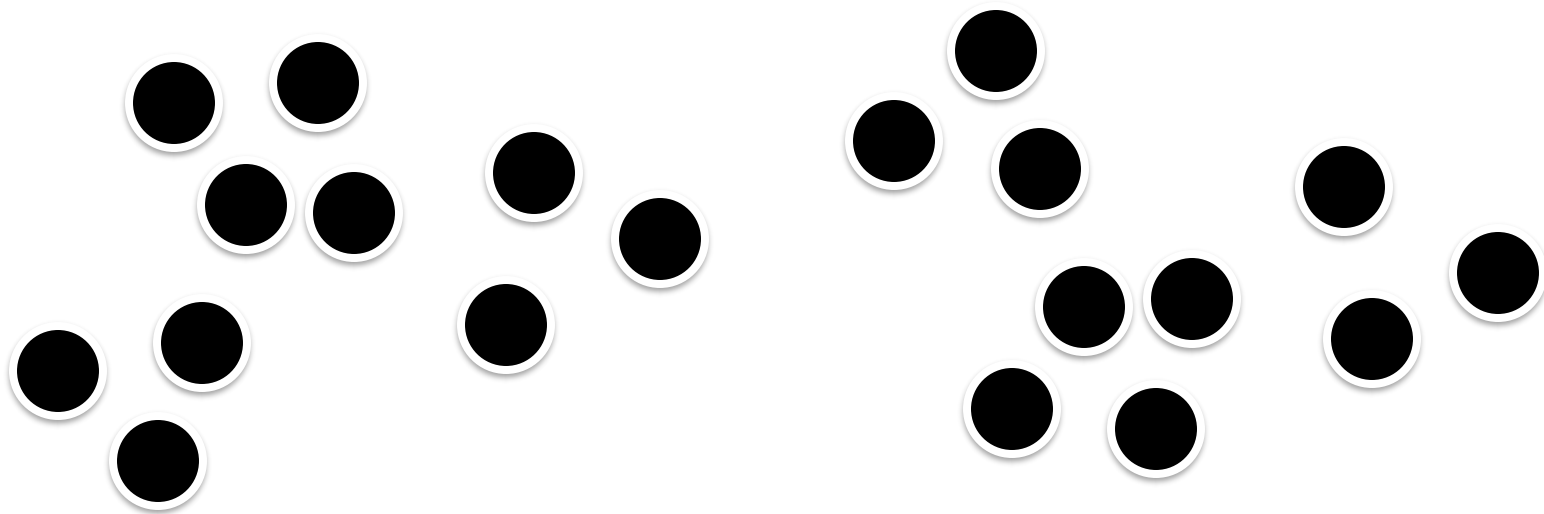


Application examples

- *Marketing :*
 - *Helps discover customer groups to develop marketing strategies;*
- *Earthquake forecasting:*
 - *The observation of seismological epicentres makes it possible to identify common segments of continental faults;*
- *Insurance companies:*
 - *Identification of user groups that represent a greater risk of contracting;*
- *Banks:*
 - *identification of customer categories (economic, social, etc.).*

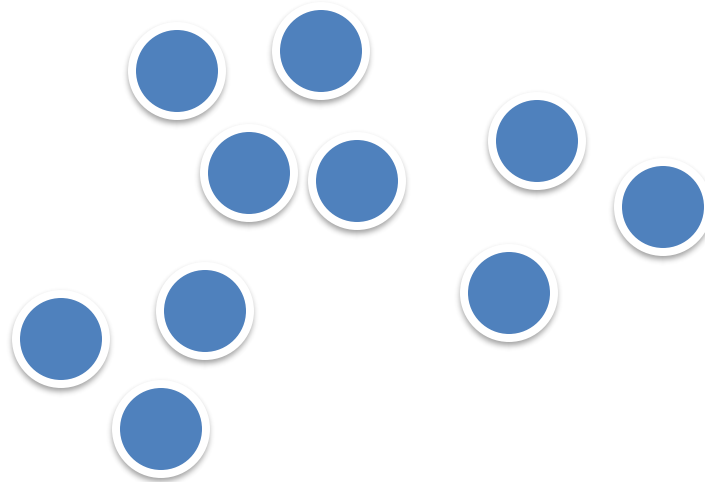


- The notion of segment is ambiguous:

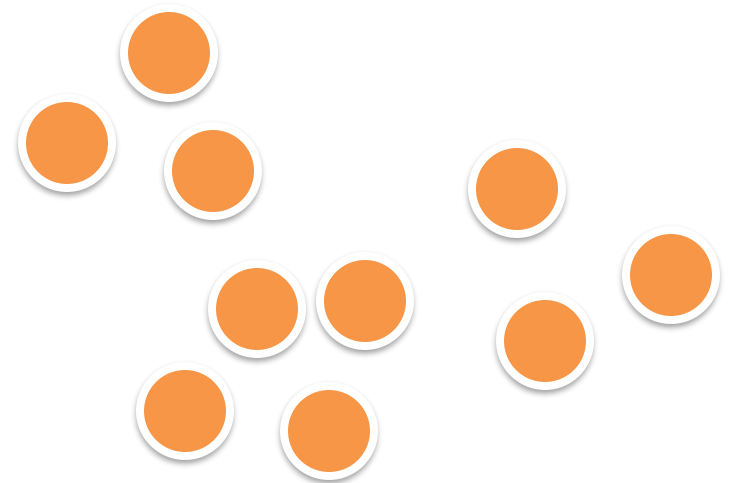


Starting points

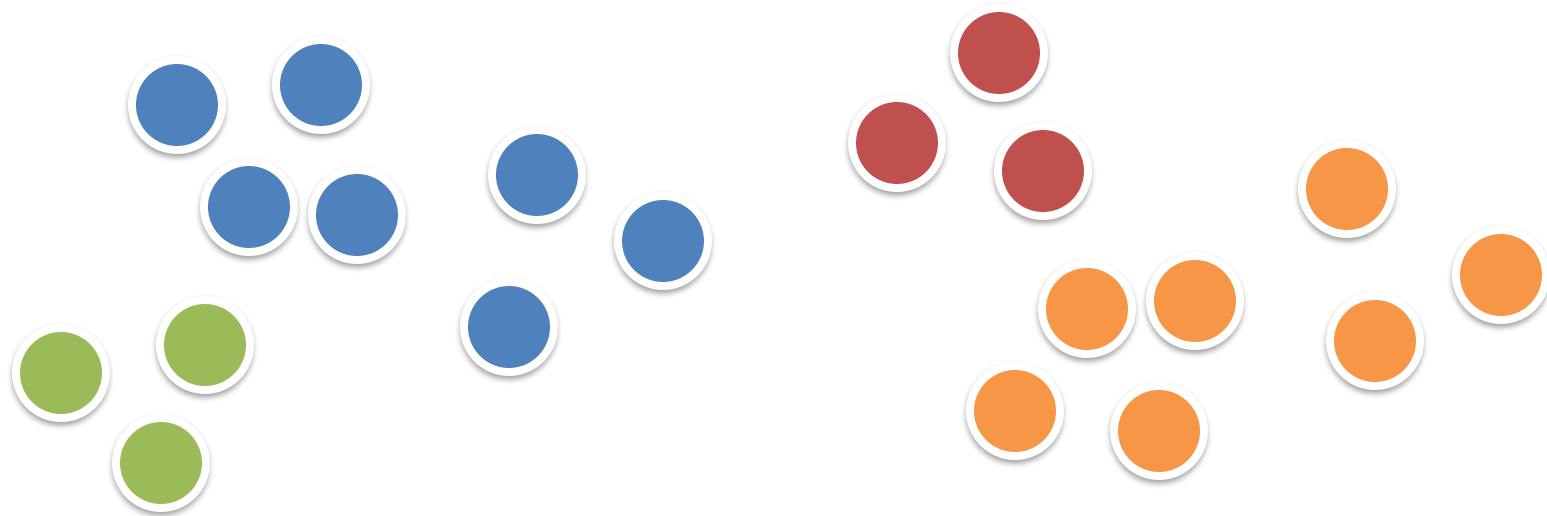
- The notion of segment is ambiguous:



2 segments

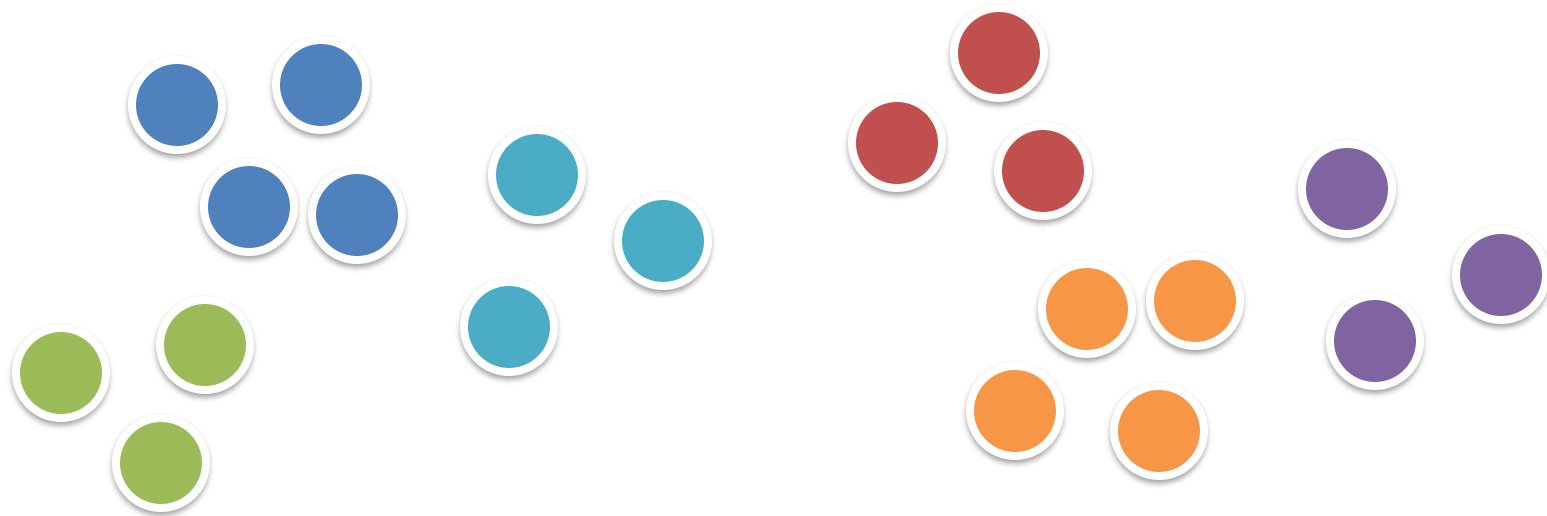


- The notion of segment is ambiguous:



4 segments

- The notion of segment is ambiguous:



6 segments

Types of data to be analysed

- Data matrix: represents 'n' objects with 'p' attributes;

X_{11}	...	X_{1j}	...	X_{1p}
...		...		...
X_{i1}	...	X_{ij}	...	X_{ip}
...		...		...
X_{n1}	...	X_{nj}	...	X_{np}

- Distance matrix: measures the proximity between pairs of objects;
- The more similar they are, the closer they are to 0 (zero).

0				
$d(2,1)$	0			
$d(3,1)$	...	0		
...	...	...	0	
$d(n,1)$	$d(n,2)$	...	...	0

Types of data to be analysed

- Continuous attributes;
- Binary attributes;
- Nominal attributes;
- Ordinal attributes;
- Mixed attributes.



Types of data to be analysed

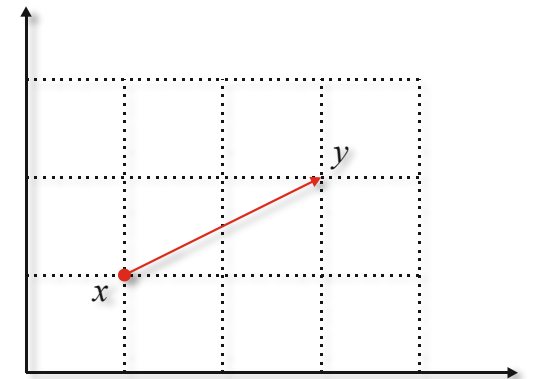
- Continuous attributes:
 - normalising the data: prevents the results from depending on the units of measurement;
 - distance measures are usually used to calculate the proximity (similarity) between objects;

Types of data to be analysed

- Continuous attributes:

- normalising the data: prevents the results from depending on the units of measurement;
- distance measures are usually used to calculate the proximity (similarity) between objects;
 - Euclidean distance: this is the measure of geometric distance in space (the most commonly used):

$$d(x, y) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \text{ (for 2 dimensions)}$$



Types of data to be analysed

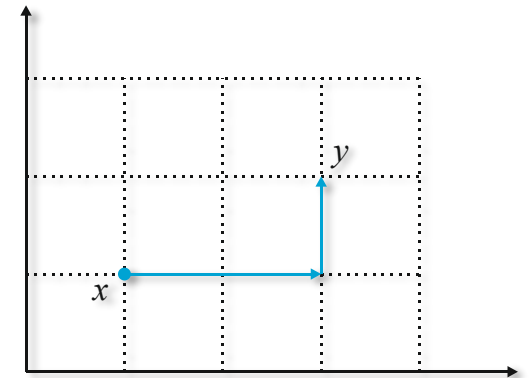
■ Continuous attributes:

- normalising the data: prevents the results from depending on the units of measurement;
- distance measures are usually used to calculate the proximity (similarity) between objects;
 - Euclidean distance: this is the measure of geometric distance in space (the most commonly used):

$$d(x, y) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \text{ (for 2 dimensions)}$$

- Manhattan distance: measures the distance by the difference between the points (non-square function):

$$d(x, y) = |x_1 - x_2| + |y_1 - y_2| \text{ (for 2 dimensions)}$$



Types of data to be analysed

- Continuous attributes:

- normalising the data: prevents the results from depending on the units of measurement;
- distance measures are usually used to calculate the proximity (similarity) between objects;
 - Euclidean distance: this is the measure of geometric distance in space (the most commonly used):

$$d(x, y) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} \text{ (for 2 dimensions)}$$

- Manhattan distance: measures the distance by the difference between the points (non-square function):

$$d(x, y) = |x_1 - x_2| + |y_1 - y_2| \text{ (for 2 dimensions)}$$

- Minkowski distance: measures the progressive weight as a function of the distance between the points :

$$d(i, j) = \left(|x_{i1} - x_{j1}|^2 + |x_{i2} - x_{j2}|^2 \right)^{1/2} \text{ (for 2 dimensions)}$$

((it's a generalisation of the previous two).

$$d(i, j) = \left(\sum_{k=1}^n |x_{ik} - x_{jk}|^p \right)^{\frac{1}{p}} \text{ (for n dimensions, } c/ p \geq 1)$$

Types of data to be analysed

- Binary attributes:
 - are classified as:
 - **Symmetrical:** the meaning of being 0 is the same as being 1;
 - **Asymmetric:** the meaning of being 0 is different from being 1;
 - the similarity calculated on the basis of symmetrical attributes is called invariant similarity; in the opposite case it is called non-invariant similarity;

Types of data to be analysed

- Binary attributes:

- are classified as:
 - **Symmetrical:** the meaning of being 0 is the same as being 1;
 - **Asymmetric:** the meaning of being 0 is different from being 1;
- the similarity calculated on the basis of symmetrical attributes is called invariant similarity; in the opposite case it is called non-invariant similarity;
- contingency table for binary data:

- simple coefficient (symmetrical):

$$d(i, j) = \frac{b + c}{a + b + c + d}$$

- Jaccard coefficient (asymmetric):

$$d(i, j) = \frac{b + c}{a + b + c}$$

	Sex	Fever	Cough	Pain
John	M	Yes	No	No
Mary	F	Yes	No	Yes
Paul	M	Yes	No	No

Types of data to be analysed

■ Binary attributes:

- are classified as:
 - **Symmetrical:** the meaning of being 0 is the same as being 1;
 - **Asymmetric:** the meaning of being 0 is different from being 1;
- the similarity calculated on the basis of symmetrical attributes is called invariant similarity; in the opposite case it is called non-invariant similarity;
- contingency table for binary data:
 - simple coefficient (symmetrical):

$$d(i, j) = \frac{b + c}{a + b + c + d}$$

- Jaccard coefficient (asymmetric):

$$d(i, j) = \frac{b + c}{a + b + c}$$

	Sex	Fever	Cough	Pain
John	M	Yes	No	No
Mary	F	Yes	No	Yes
Paul	M	Yes	Yes	No

		Mary		
John	Sex	M	F	Sum
	M	a = 0	b = 1	a+b
	F	c = 0	d = 0	c+d
	Sum	a+c	b+d	

Types of data to be analysed

■ Binary attributes:

- are classified as:
 - **Symmetrical:** the meaning of being 0 is the same as being 1;
 - **Asymmetric:** the meaning of being 0 is different from being 1;
- the similarity calculated on the basis of symmetrical attributes is called invariant similarity; in the opposite case it is called non-invariant similarity;
- contingency table for binary data:
 - simple coefficient (symmetrical):

$$d(i,j) = \frac{b + c}{a + b + c + d}$$

- Jaccard coefficient (asymmetric):

$$d(i,j) = \frac{b + c}{a + b + c}$$

	Sex	Fever	Cough	Pain
John	M	Yes	No	No
Mary	F	Yes	No	Yes
Paul	M	Yes	Yes	No

		Mary		
John	F/G/P	S	N	Sum
	S	a = 1	b = 0	a+b
	N	c = 1	d = 1	c+d
	Sum	a+c	b+d	

Types of data to be analysed

- Nominal attributes:
 - This is a generalisation of binary attributes, where the data can take on more than 2 values;
 - Method 1:
 - *matching* simples;
 - $d(i, j) = \frac{n^{\circ}variables - n^{\circ}matches}{n^{\circ}variables}$
 - Method 2:
 - Use binary variables;
 - Create a binary variable for each nominal value.

Types of data to be analysed

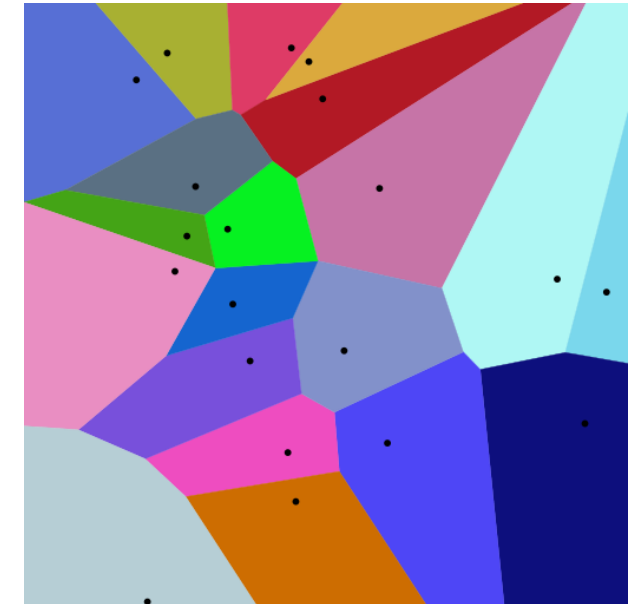
- Ordinal attributes:
 - the order is relevant:
 - first, second, third, ..., penultimate, last;
 - can be treated as continuous attributes, with the ordering of the values defining a classification:
 - 1, 2, 3, ..., Max;
 - similarities must be calculated using the same methods as for continuous attributes.

Types of data to be analysed

- Continuous attributes;
- Binary attributes;
- Nominal attributes;
- Ordinal attributes;
- Mixed attributes:
 - the data set can contain different types of attributes;
 - Typically, a weighted function is used to weight and measure the effects of each attribute.

Main Segmentation Methods

- Partitioning:
 - create several partitions and adopt an evaluation criterion;
- Hierarchisation:
 - Hierarchically decompose the data set;

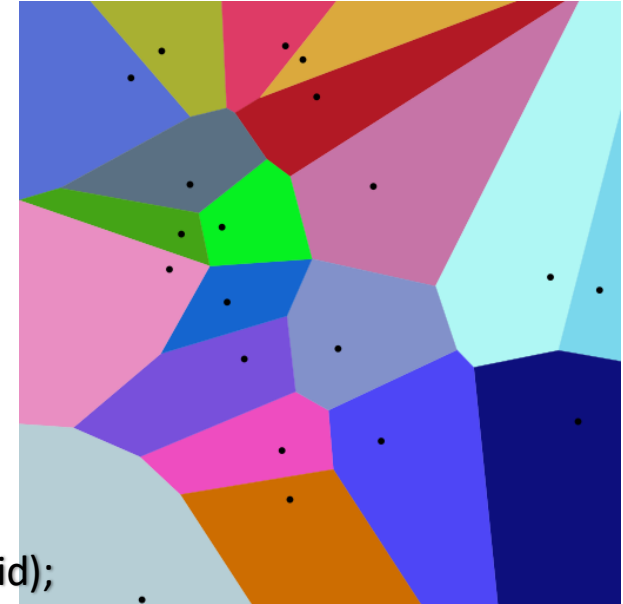


Main Segmentation Methods

- Partitioning:
 - create several partitions and adopt an evaluation criterion;
- Hierarchisation:
 - Hierarchically decompose the data set;
- Others:
 - Density-based:
 - Increase the segment as long as the density of points is within a certain limit (connectivity and density functions are used);
 - Model-based:
 - create hypothetical models for each segment and test the suitability of each point to the segment.

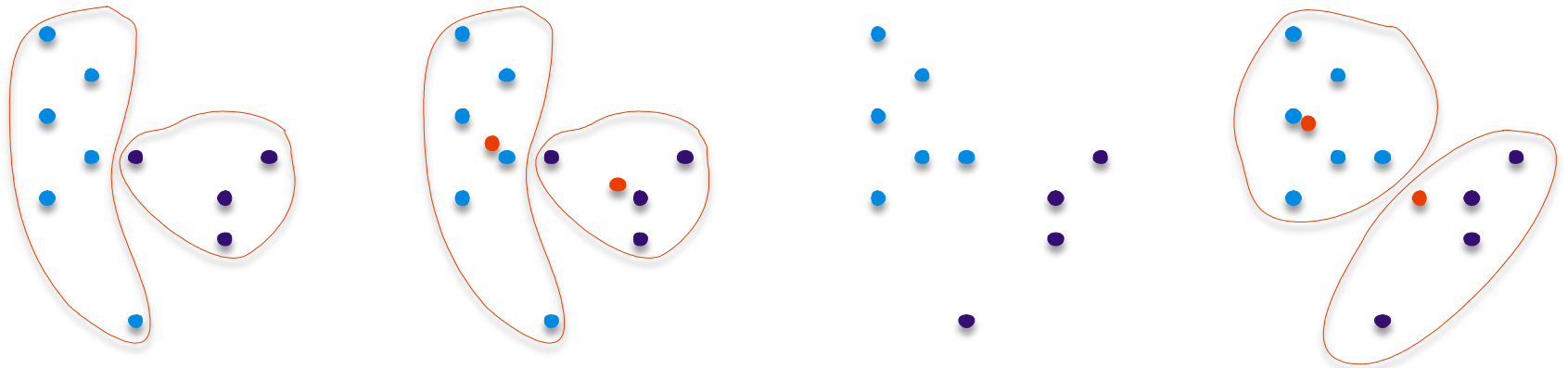
- Partition a dataset 'D' containing 'n' objects into a set of 'k' segments/clusters;
- Given 'k', partition 'D' into 'k' segments in order to optimize the partitioning criterion:
 - Global optimum: exhaustive enumeration of all partitions;
 - Heuristic methods:
 - k-means: each segment is represented by the centre of the segment (centroid);
 - k-medoids: each segment is represented by one of the elements of the segment (medoid).

Partition Algorithms



Partition Algorithms k-means method

- Given 'k' (number of segments), follow the 4 steps:
 - Divide the objects into 'k' non-empty subsets;
 - Calculate the centre of each segment (centroid);
 - Assign each object to the nearest centroid;
 - Go back to point 2; stop when there are no more assignment possibilities.

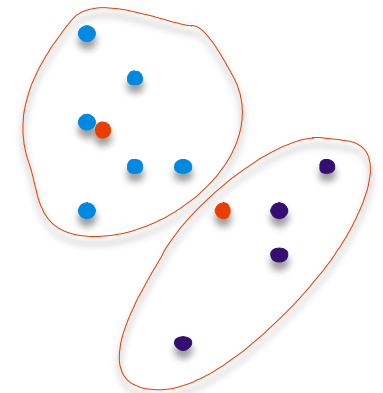
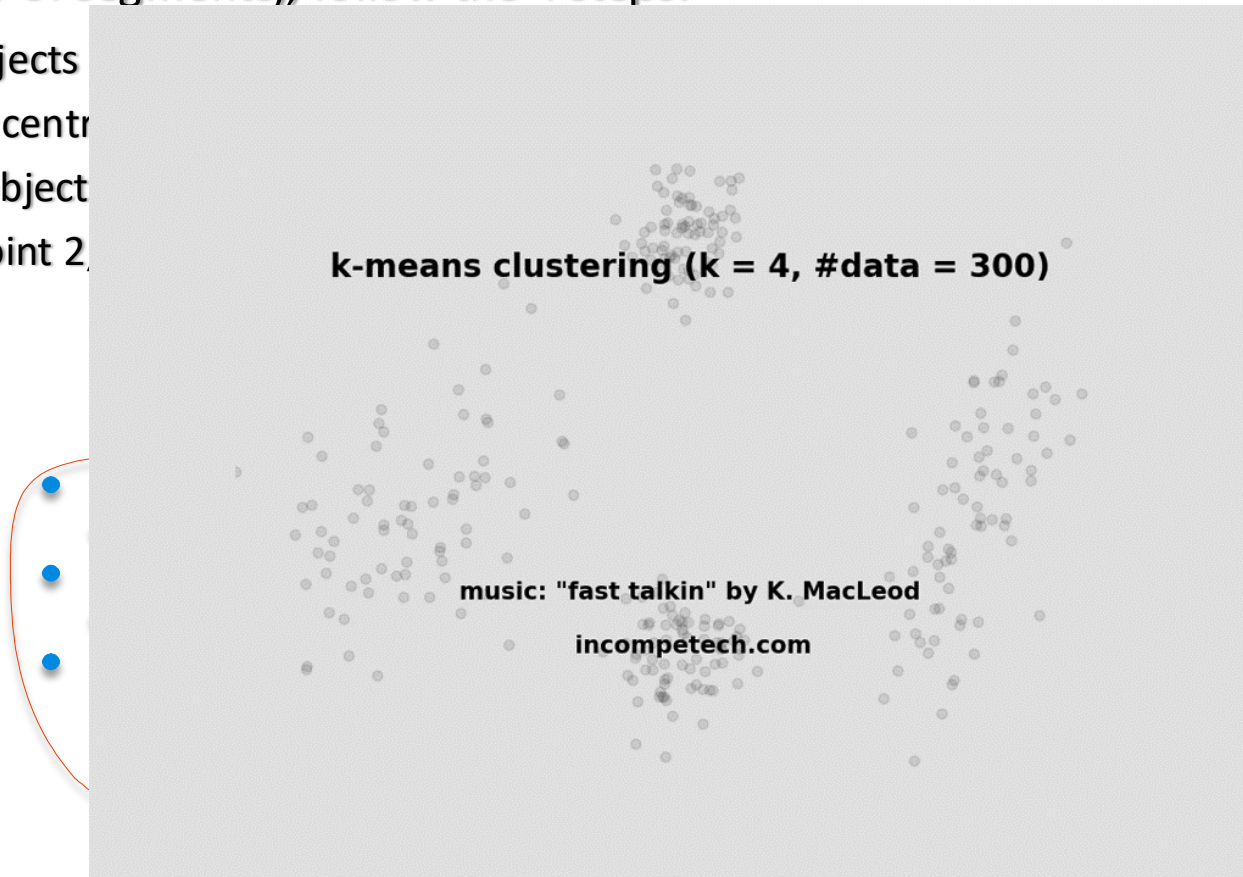


Partition Algorithms

k-means method

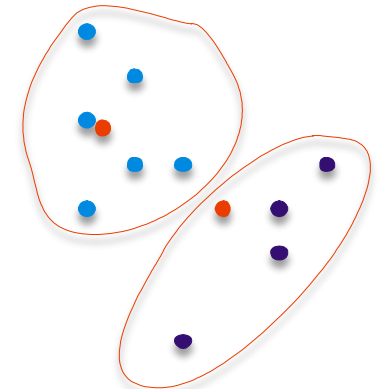
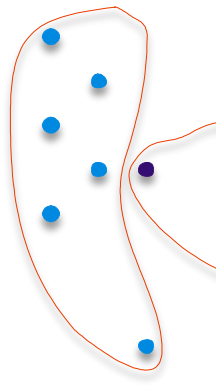
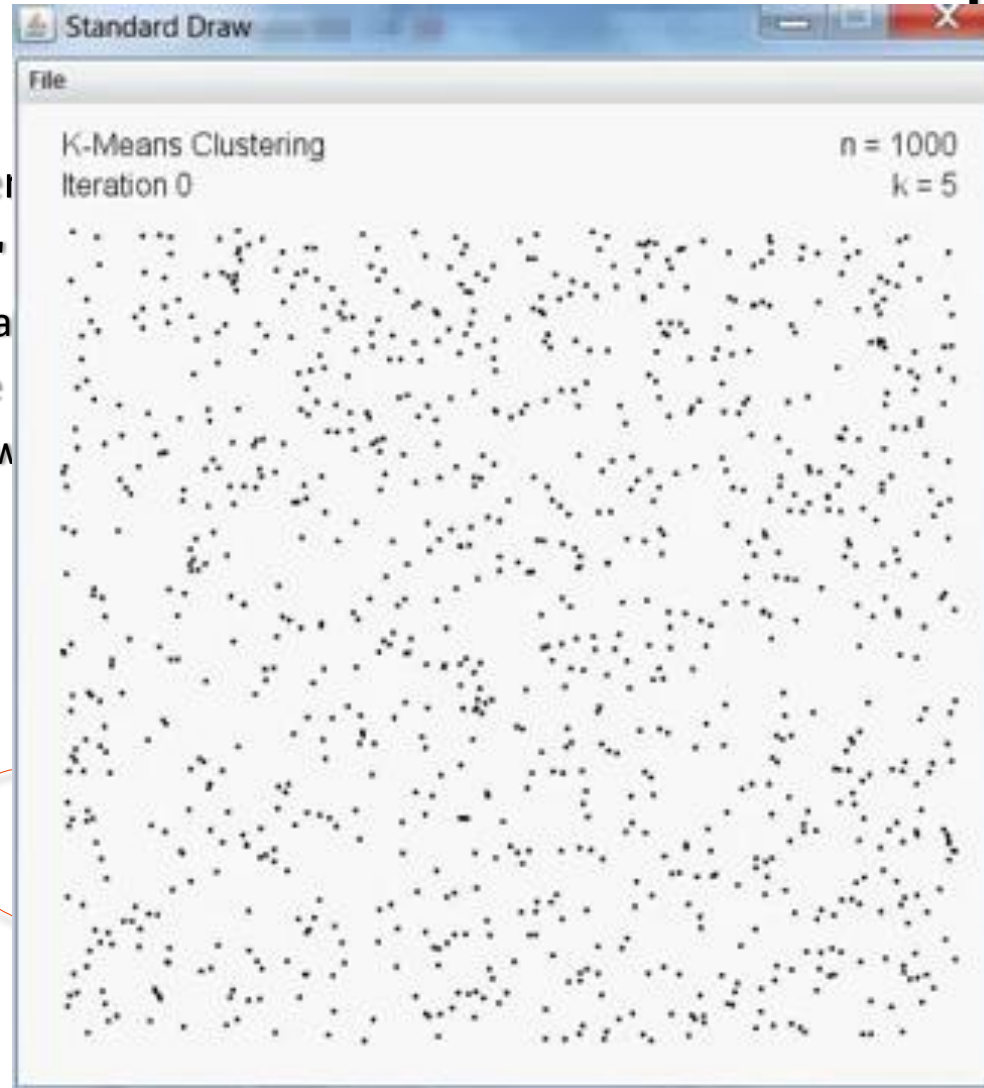
- Given 'k' (number of segments), follow the 4 steps:

- Divide the objects
- Calculate the centre
- Assign each object
- Go back to point 2



Partition Algorithms k-means method

- Given 'k' (number of segments)
 - Divide the objects into 'k'
 - Calculate the centre of ea
 - Assign each object to the
 - Go back to point 2; stop w



k-means method (example by hand)

- We start with 9 objects that we want to divide into 2 segments;



k-means method (example by hand)

- We started with a random positioning of 2 centroids;



k-means method (example by hand)

- We measure the distance from each object to each centroid to determine which one is closest;



k-means method (example by hand)

- We assign the first object to the segment represented by the nearest centroid;



k-means method (example by hand)

- We make the same comparison for all the (remaining 8) objects...



k-means method (example by hand)

- ... to associate them with the respective centroid...



k-means method (example by hand)

- ... always based on the smallest distance measuring the greatest similarity;



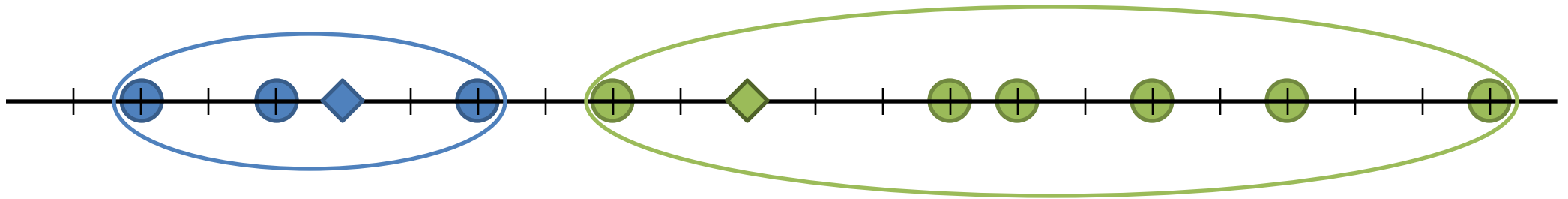
k-means method (example by hand)

- At the end of the first iteration, we have all the objects associated with its centroid...



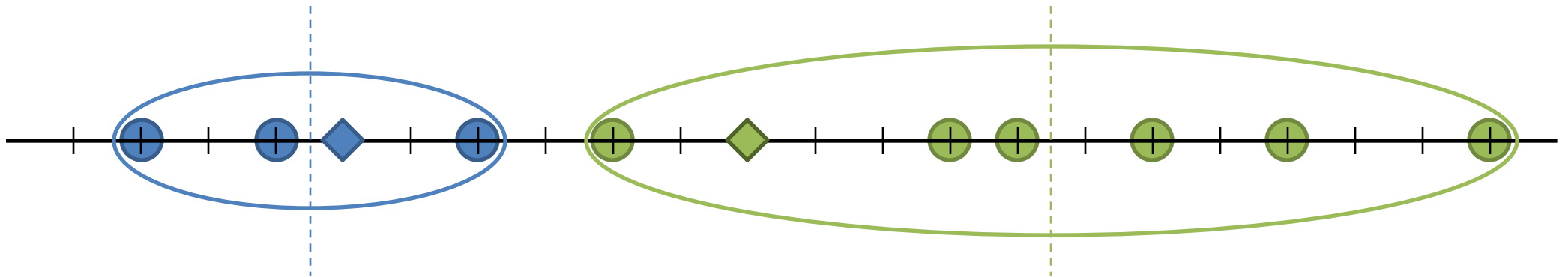
k-means method (example by hand)

- ... which identifies the first 2-segment solution;



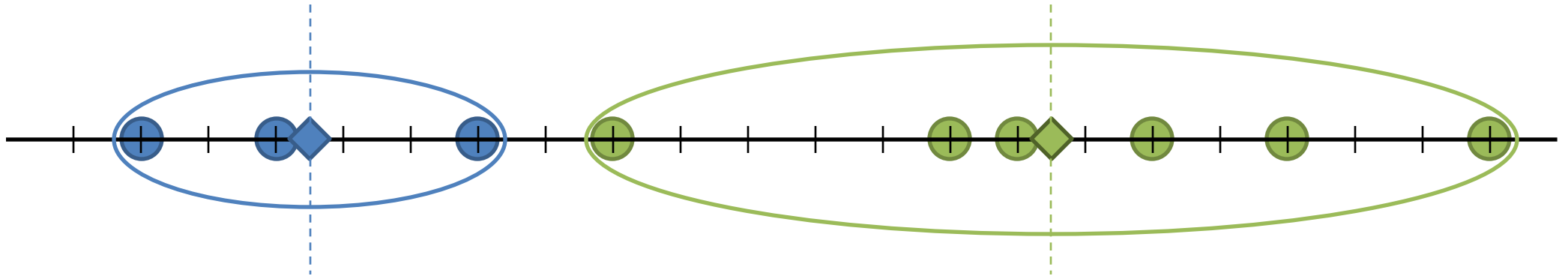
k-means method (example by hand)

- We calculate the centre of the segment...



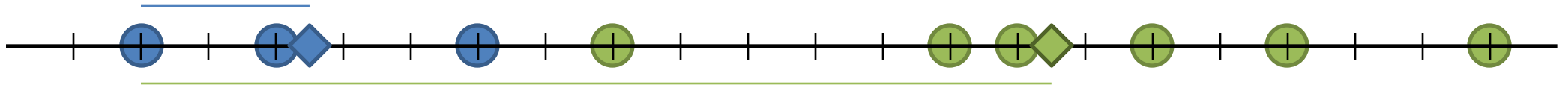
k-means method (example by hand)

- ... and we put the centroid there;



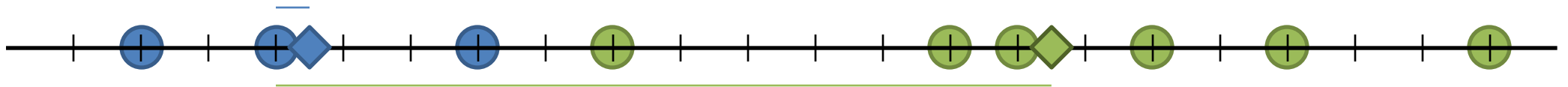
k-means method (example by hand)

- From here the process repeats...



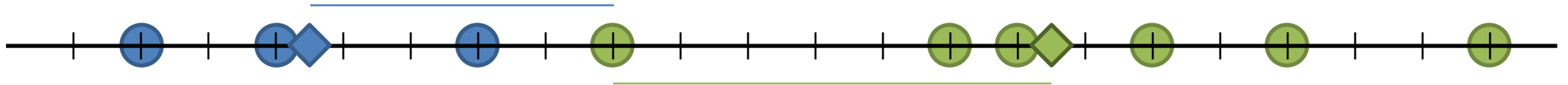
k-means method (example by hand)

- ... in the sense of reorganising the association of objects with their closest centroids;



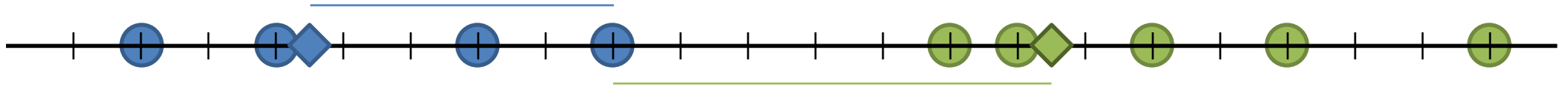
k-means method (example by hand)

- Thus, this object...



k-means method (example by hand)

- ... will move on to the other segment;



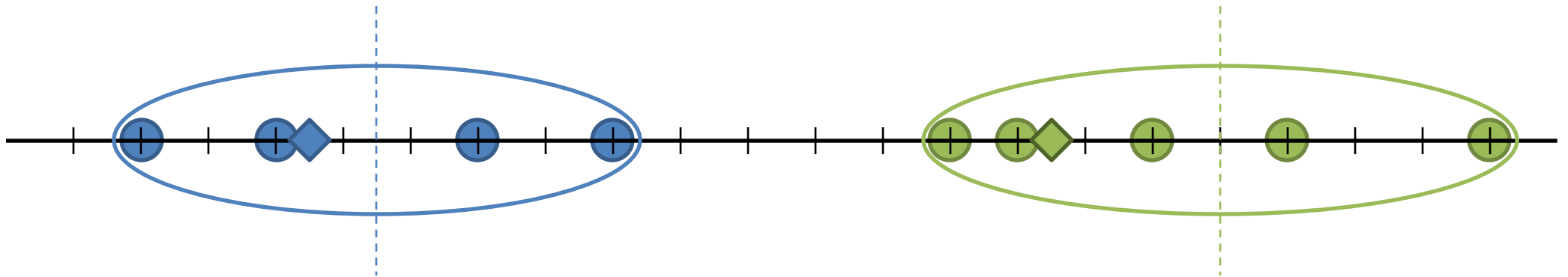
k-means method (example by hand)

- The 2 segments now have this configuration;



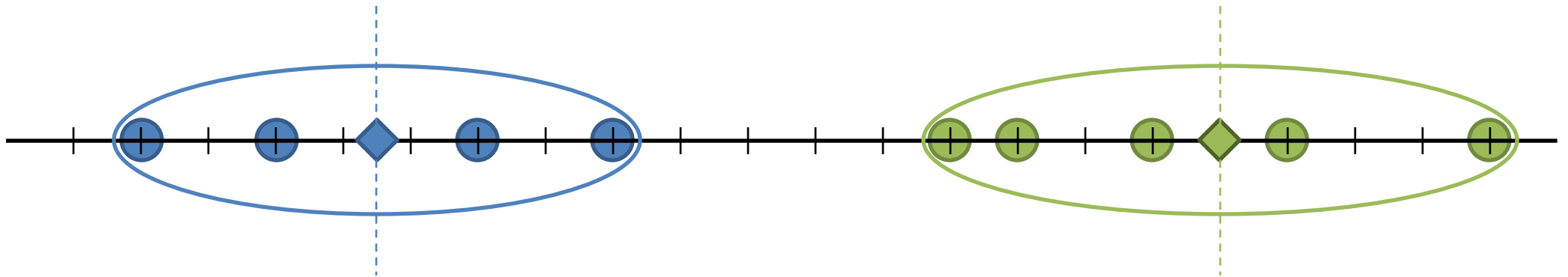
k-means method (example by hand)

- Again, we calculate the centre of each segment...



k-means method (example by hand)

- ... and replace its centroid in that position;



k-means method (example by hand)

- The process continues iteratively, associating objects with the most similar centroids;



Partition Algorithms

k-means method

■ Advantages:

- Relatively efficient: where 'n' is the number of objects, 'k' the number of segments and 'i' the number of iterations, $k, i \ll n$ usually happens;
- Ends up with local optima.

■ Disadvantages:

- Only applicable when it is possible to calculate the mean;
- It is necessary to identify the number of segments a priori;
- Inability to deal with noise in the data;
- Unsuitable for determining concave segments.



Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.

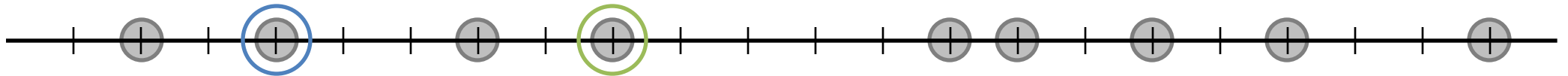
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



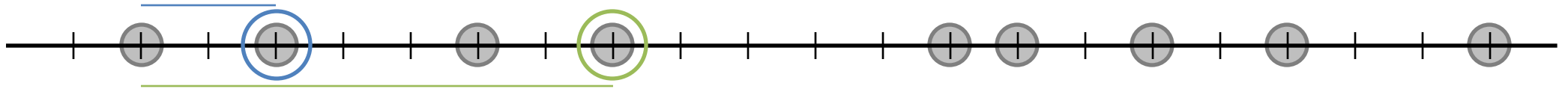
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



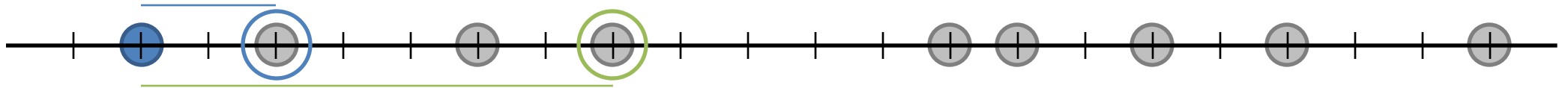
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



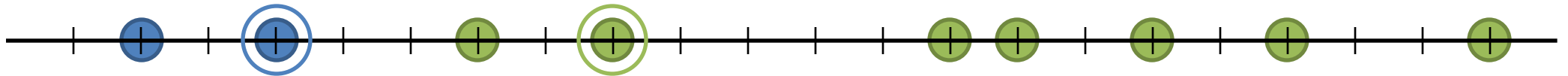
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



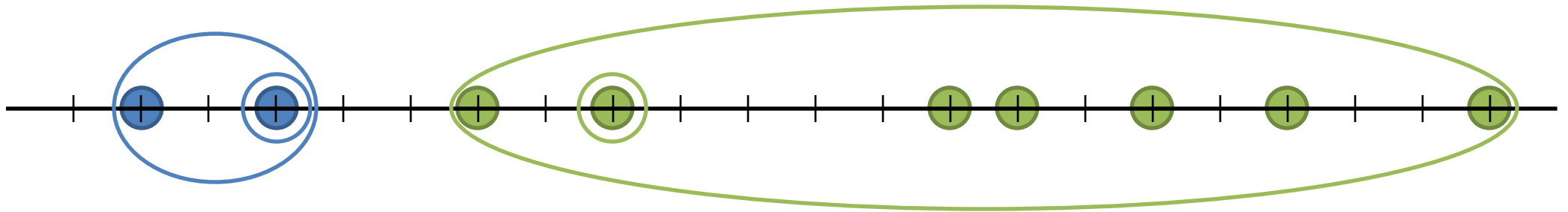
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



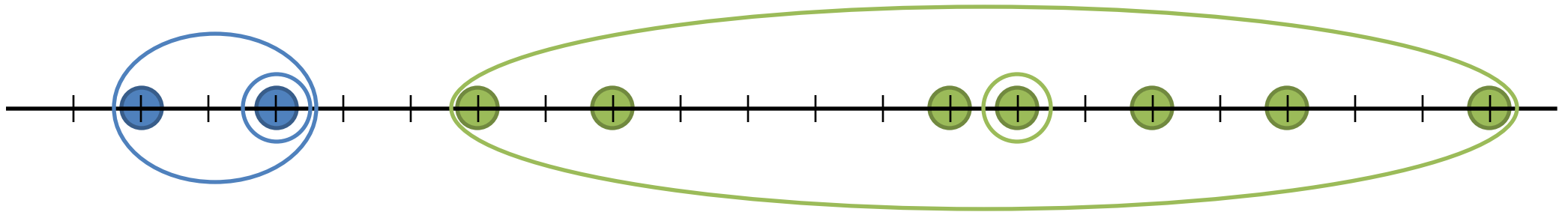
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



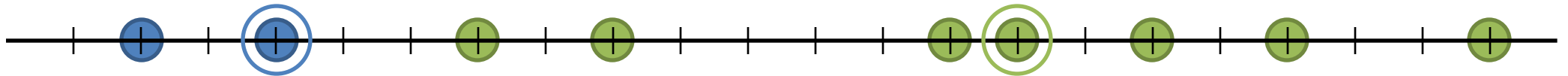
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



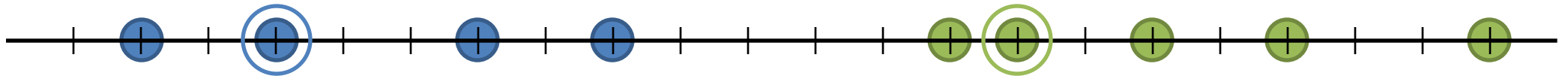
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



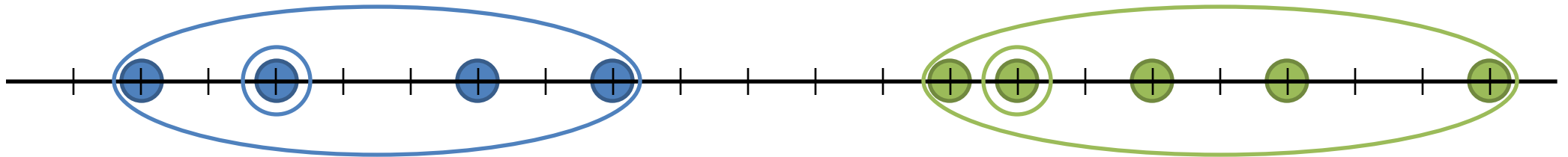
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



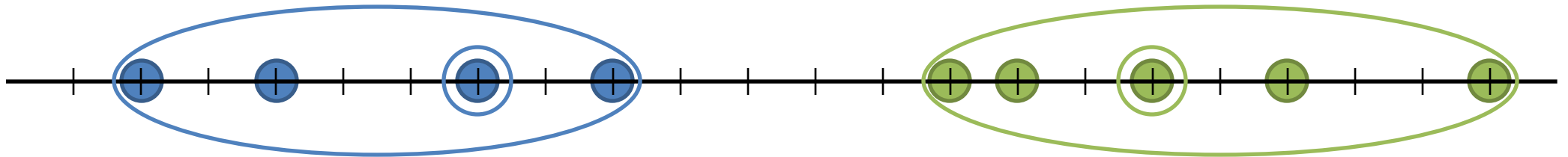
Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



Partition Algorithms k-medoids method

- Medoids are objects that are **representative** of the data set;
- It starts with a set of medoids that are iteratively replaced by other non-medoids as long as the distance of the resulting segment is improved.



Partition Algorithms k-medoids method

- Advantages and Disadvantages:

- It is more robust than the k-means method in the presence of noisy data, since the selected objects are less influenced by extreme values than the mean;
- It produces good results for small data sets;
- Does not behave as well when applied to large data sets.



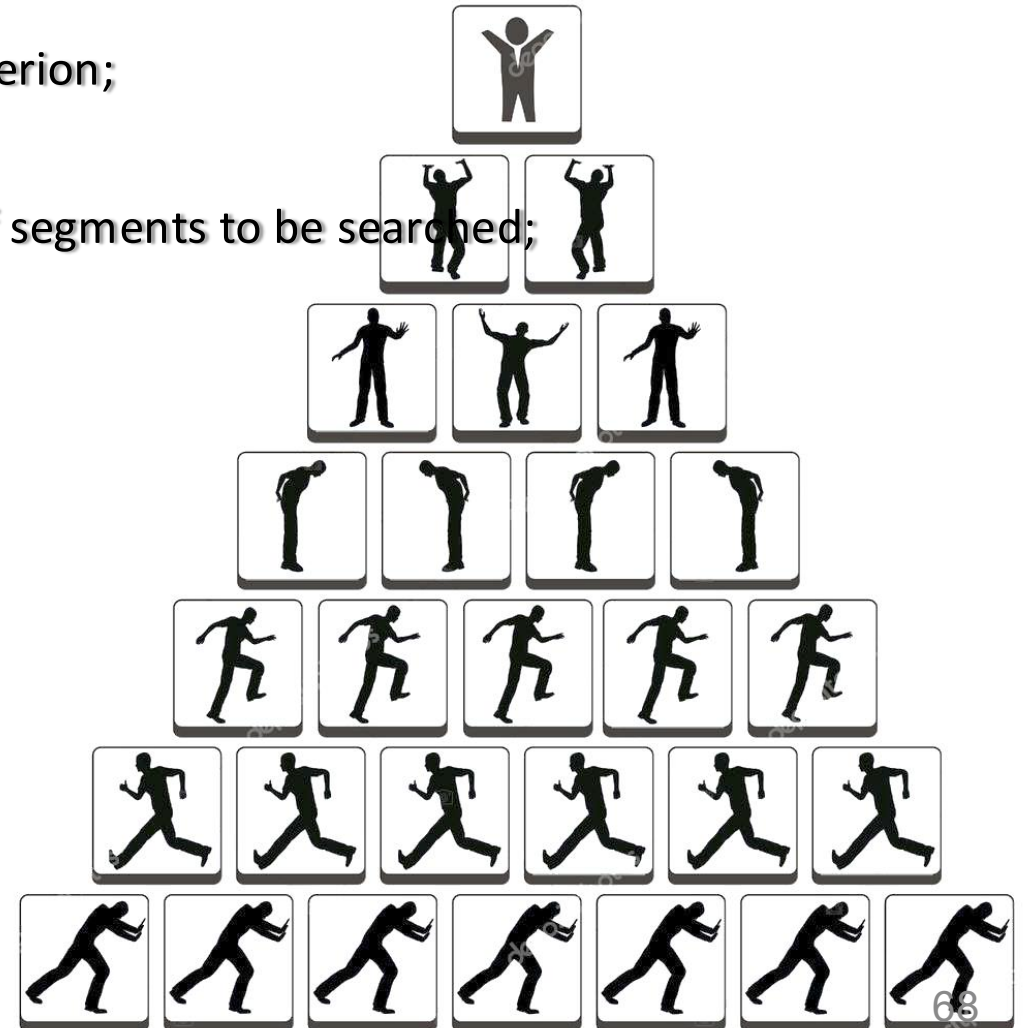
Main Segmentation Methods

- Partitioning:
 - create several partitions and adopt an evaluation criterion;
- Hierarchisation:
 - Hierarchically decompose the data set;
- Others:
 - Density-based:
 - Increase the segment as long as the density of points is within a certain limit (connectivity and density functions are used);
 - Model-based:
 - create hypothetical models for each segment and test the suitability of each point to the segment.



Hierarchisation algorithms

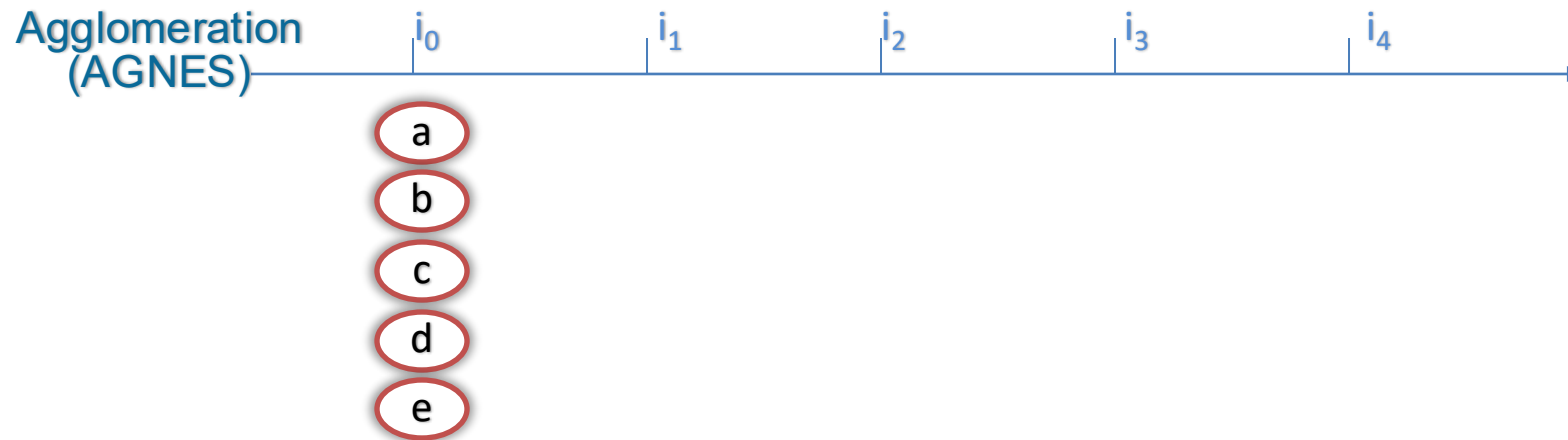
- They use the distance matrix as a segmentation criterion;
- The data is grouped into segment trees;
- They do not require the definition of the number of segments to be searched;
- They require the definition of a stop condition:
 - number of segments;
 - minimum distance between objects;
 - etc.
- There are two types of hierarchisation algorithms:
 - Agglomeration: bottom-up strategy;
 - Division: top-down strategy.



Hierarchisation algorithms

■ Agglomeration:

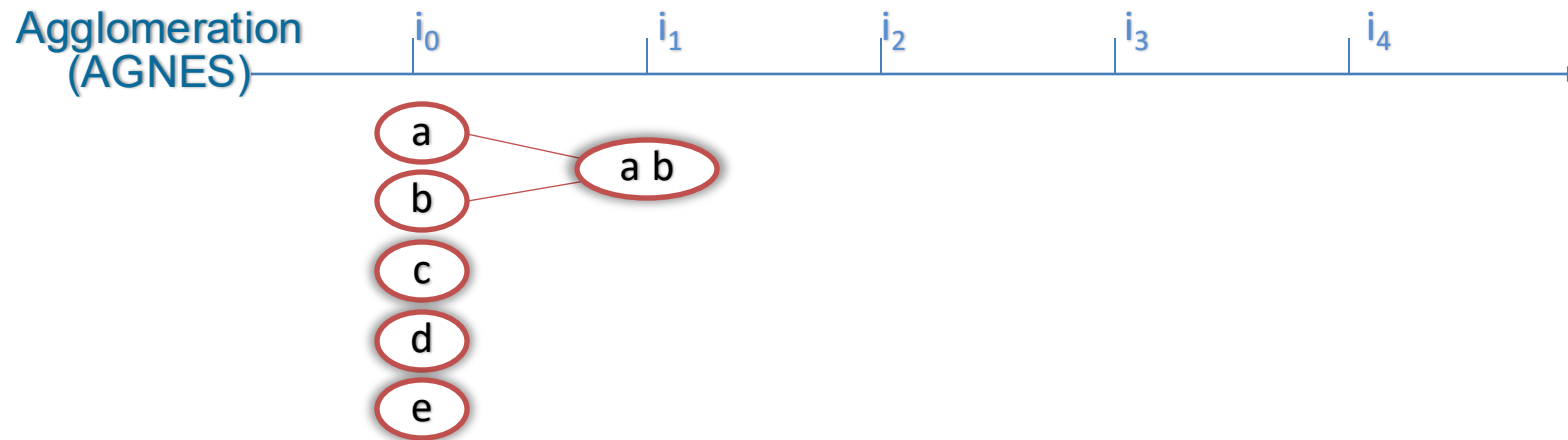
- Starts by forming segments with one object, for all objects;
- It continues by joining atomic segments into larger and larger segments.



Hierarchisation algorithms

■ Agglomeration:

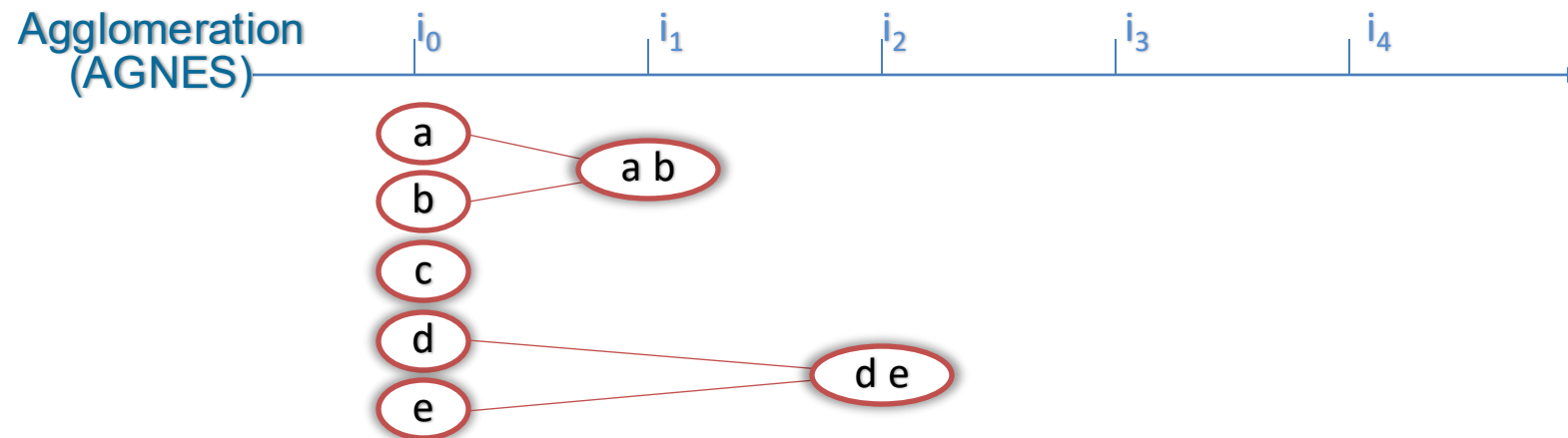
- Starts by forming segments with one object, for all objects;
- It continues by joining atomic segments into larger and larger segments.



Hierarchisation algorithms

■ Agglomeration:

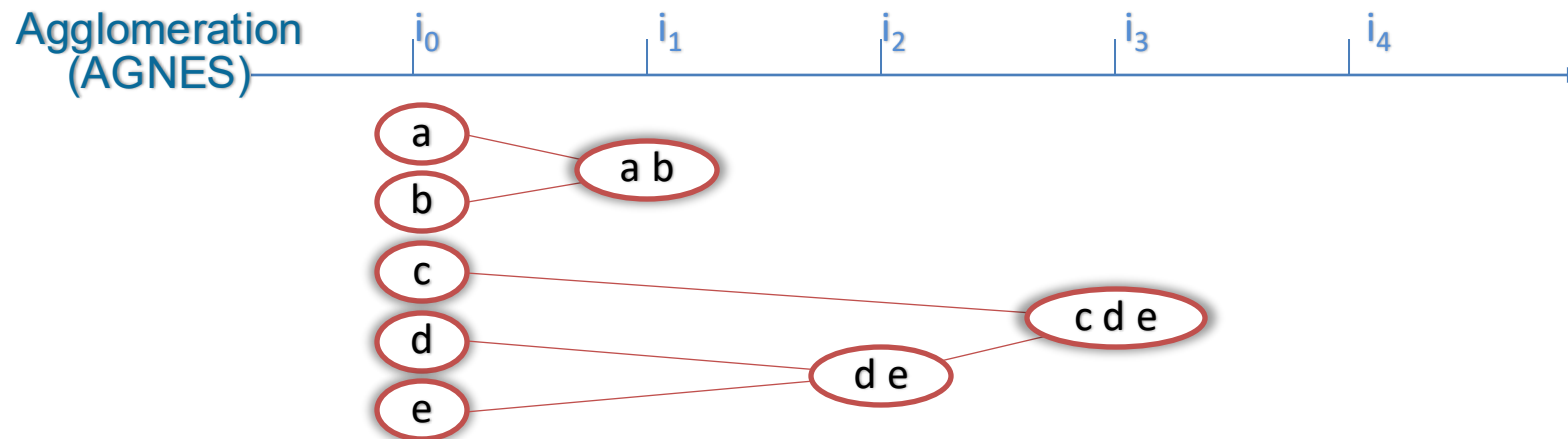
- Starts by forming segments with one object, for all objects;
- It continues by joining atomic segments into larger and larger segments.



Hierarchisation algorithms

■ Agglomeration:

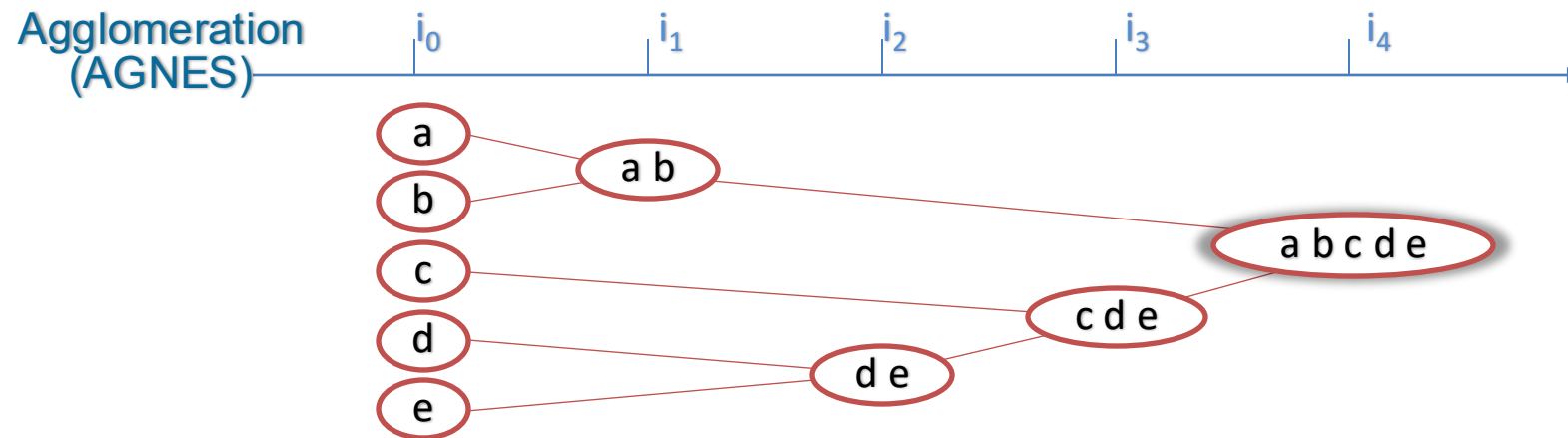
- Starts by forming segments with one object, for all objects;
- It continues by joining atomic segments into larger and larger segments.



Hierarchisation algorithms

■ Agglomeration:

- Starts by forming segments with one object, for all objects;
- It continues by joining atomic segments into larger and larger segments.



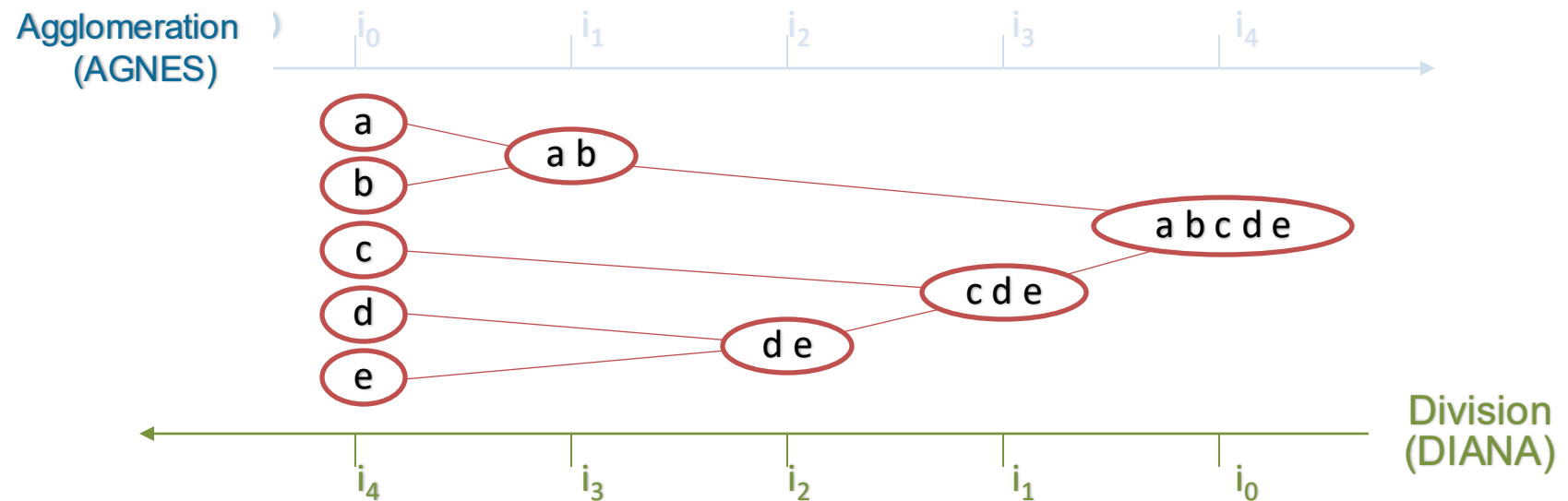
Hierarchisation algorithms

■ Agglomeration:

- Starts by forming segments with one object, for all objects;
- It continues by joining atomic segments into larger and larger segments.

■ Division:

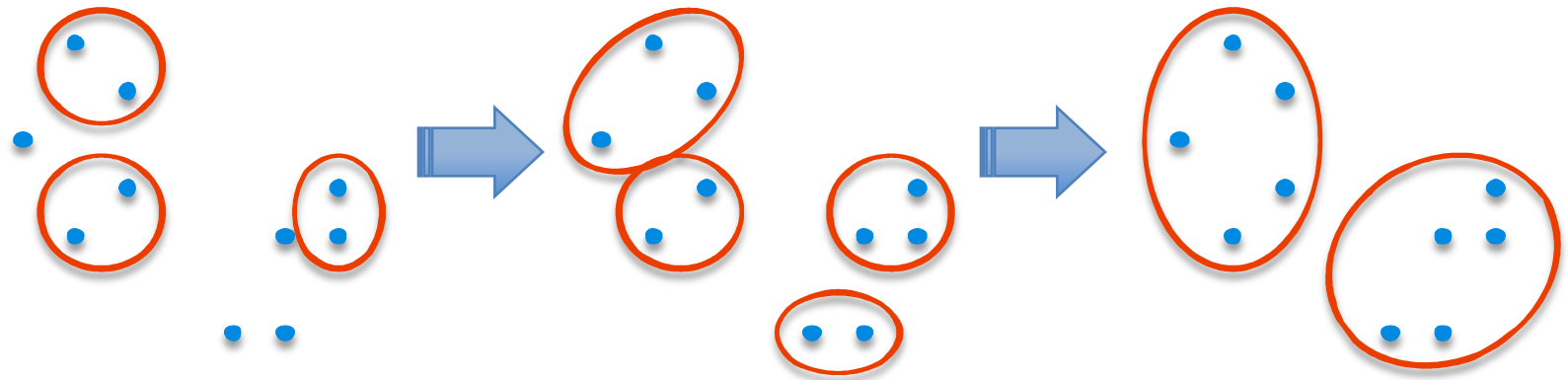
- Starts with all the objects in a single segment which is subdivided into smaller segments;
- Very rare practical application.



Hierarchisation algorithms

AGNES: Agglomerative Nesting

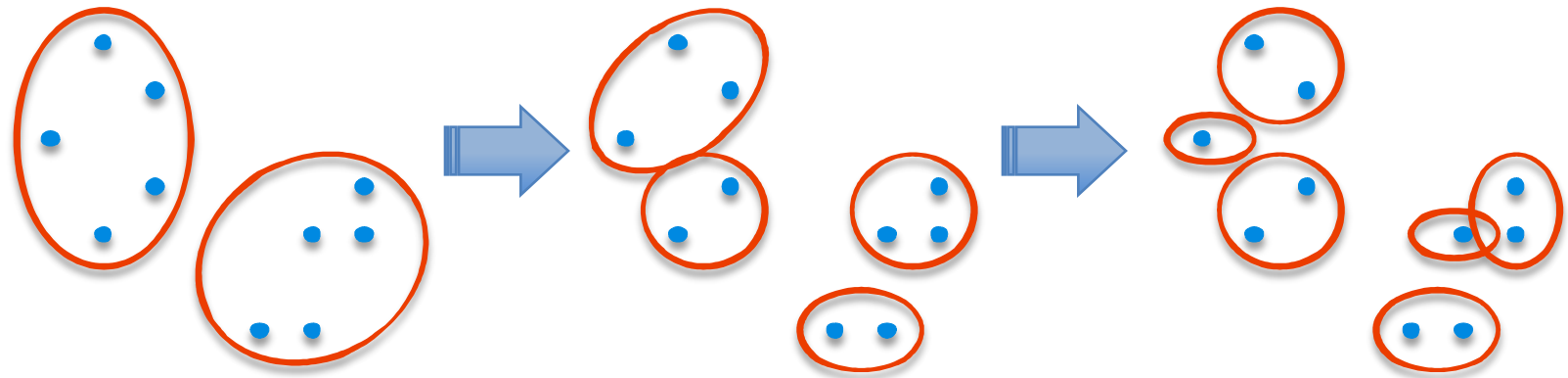
- Iteratively, it joins objects that have lower dissimilarity values: sets C1 and C2 are joined if the objects in C1 and C2 produce the lowest Euclidean distance value between any two objects in different segments.



Hierarchisation algorithms

DIANA: Divisive Analysis

- Iteratively and starting from a segment made up of all the objects, divide it into smaller segments that maximise the Euclidean distance between neighbouring objects in different segments.



Hierarchical segmentation

- Difficulties with increasing attributes or objects:
 - as the number of objects to group increases, the time needed to search for these groups increases;
- No need to specify the number of 'k' segments; simply "cut" the tree at the 'k-1' level;
- Produces better results than k-means algorithms;
- A hierarchy reflects some organisation of the segments, as opposed to a simple set of segments.



Other algorithms

BIRCH: Balanced Iterative Reducing and Clustering using Hierarchies;

- It uses trees with characteristics about the segments and iteratively adjusts the quality of the segments;
- A tree is built that captures the information needed to carry out the segmentation operations:
 - Clustering Feature: contains information about the segment;
 - Clustering Feature Tree: contains information about the tree organisation of the hierarchy.

Other algorithms

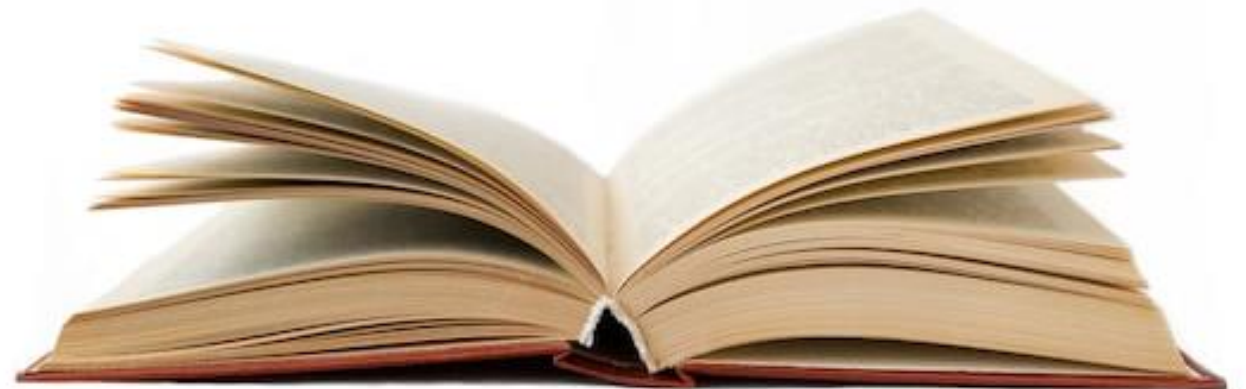
CURe: Clustering Using Representatives;

- It selects scattered points on the segment and reduces the size of the segment towards its centre;
- It uses multiple representative points;
- In each iteration, two segments with the closest pair of representative points are joined together.

Other algorithms

- DBSCAN: *Density Based Spatial Clustering of Applications with Noise*;
- Algorithm based on calculating local density and connectivity values;
- Notable features:
 - able to discover segments of non-regular shapes;
 - capable of dealing with noise in the data;
 - one-step algorithm (scan);
 - requires the definition of density parameters as a stop condition.

- Data Mining: Concepts and Techniques
Jiawei Han, Micheline Kamber
- Data Mining: Practical Machine Learning Tools and Techniques with JAVA
Implementations
Ian Witten, Eibe Frank





Universidade do Minho

Escola de Engenharia

Departamento de Informática

APRENDIZAGEM AUTOMÁTICA

Machine Learning

Clustering

MESTRADO EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA

MESTRADO integrado EM ENGENHARIA DE TELECOMUNICAÇÕES E INFORMÁTICA

Inteligência Artificial para as Telecomunicações

2025/2026